

COMP 6721 - Final report - Group A

Abstract

Malware detection is a critical task as cyber threats continue to evolve, challenging traditional detection methods. This project leverages deep learning based automated classification using Portable Executable files in byteplot image format, enhancing efficiency and reducing computational complexity. ResNet-18, EfficientNet-B0, and DenseNet-121 were employed to analyze malware image datasets of varying sizes. Data preprocessing steps included image resizing, class balancing, and normalization, ensuring robust model training. Models were then evaluated using accuracy, precision, recall, F1-score, confusion matrix, and AUC-ROC to measure overall performance and minimize false positives/negatives. Results demonstrate the effectiveness of using byteplot images for malware detection, achieving high accuracy and still keeping resource efficiency. Even though this solution has its disadvantages, this approach still offers a scalable and adaptive solution to modern malware detection challenges.

A. Introduction

Malware detection has become increasingly critical as the frequency and sophistication of digital threats continue to rise. The traditional approaches of malware detection, such as block lists for file extensions, static signatures, and dynamic analysis are limited in their ability to keep up with the evolving tactics used by attackers [1]. Malware authors frequently update their techniques, rendering conventional methods less effective over time [2]. As a result, modern malware detection requires more advanced and adaptive technologies that can anticipate and respond to these changes without relying solely on predefined rules. In this situation, Artificial intelligence (AI), particularly deep learning, offers a transformative approach [3].

Nowadays, research regarding deep learning for malware detection can be divided into four areas which are behavior analysis detection, anomaly detection, signature-based detection, and automated classification[4]. Behavior analysis focuses on using deep learning to detect malware actions and behavioral patterns on machines. Anomaly detection focuses on using deep learning to create a model of "normal" behavior to highlight potential threats[4]. Signature-based detection uses deep learning to distinguish possible malware files with signatures. Automated classification uses deep learning to automate the detection and classification of possible malware files. Even though automated classification has been proven that it can detect malware efficiently[5], there still exists one issue regarding it, which is automated classification requires a lot of resources to

deal with complex data. In this project, we focus on automated classification using Portable Executable(PE) files in byteplot image format, which helps prevent malware initially and reduces the workload for other methods. Furthermore, by converting binary files into structured byteplots, we can reduce data complexity and make the classification process more resource-efficient. This approach has great potential to resolve the computational burden typically associated with malware detection.

In this project, we developed a malware detection system using CNNs (ResNet-18, EfficientNet-B0, and DenseNet-121) to analyze image-based representations of malware files, leveraging PE byteplot images. We trained and evaluated these models on three malware datasets, using data preprocessing techniques to balance classes and ensure consistent image quality. Our training involved systematic tuning of hyper-parameters, with the Adaptive moment estimation(Adam) optimizer and cross-entropy loss yielding strong performance. Regarding evaluating models, we used accuracy to measure overall performance, precision to assess the proportion of true positives, and recall to determine how well the models identify actual malware. F1-score provides a balanced view by combining precision and recall. The confusion matrix gave detailed insights into performance across all classes. We also used AUC-ROC(Area Under the Receiver Operating Characteristic Curve) to evaluate class discrimination and monitor false positive rates(FPR) and false negative rates(FNR) to ensure the system minimizes miss classifications. These metrics guided us in developing an efficient and accurate malware detection system.

B. Methodology

B.1. Dataset

In this project, we aim to leverage Convolutional neural networks(CNNs) to develop a malware detection system that analyzes image-based representations of malware files. By using Portable Executable (PE) files in byteplot image format, our approach allows CNNs to interpret malware data. CNNs are excellent at identifying image patterns, making it possible for the model to detect subtle similarities and differences in samples that traditional methods may miss. To train and evaluate this system, we employed three diverse malware image datasets from Kaggle. The large dataset is the Blended Malware Image Dataset [6], which includes 9,868 PNG images across 31 malware classes, each at a resolution of 224x224 pixels. Secondly, we are using the Malware Images dataset[7] as the medium dataset, containing 125,004 images in 17 classes. Although it is skewed toward a few specific classes, preprocessing

steps will help balance it as our moderate dataset. Finally, the Malware-Byteplot Images dataset[8], offers a binary classification of 60,337 images labeled as "Benign" and "Malicious." This dataset, although slightly skewed towards the malicious class, will serve as our small dataset.

As mentioned in the previous paragraph, some datasets are skewed toward a few specific classes. To address this and prevent it from negatively impacting the training process and final CNN performance, this project incorporates a data preprocessing pipeline. The first step involves resizing all images to 224x224, ensuring consistent image dimensions and avoiding errors related to varying sizes. The second step is to calculate the sample count per class, find the average number of samples across all classes, and then balance the sample count between classes. This balancing is achieved using undersampling and oversampling. If some of the classes have more samples than the average number of samples, the pipeline will perform undersampling by selecting a random subset of the images to match the target count. If some of the classes have fewer samples than the average number of samples, the pipeline will perform oversampling by randomly duplicating a subset of the images and performing augmentation action until the target count is reached. Augmentation includes random rotation, width shift, height shift, random adjustments on the brightness range, and random horizontal flip. Balancing the dataset through these augmentations helps prevent poor generalization, especially for minority classes, and ensures that models can learn from a more representative distribution of data[9]. The third step of this pipeline is to normalize all sample image pixel values from between 0 and 255 to between 0 and 1. This normalization helps reduce computation time during training and reduces variability, leading to more consistent activations and gradient distributions across the network[10]. The last step is to split all samples into three sets i.e., Training, Validation, and Testing. For small and medium datasets, the percentage is 70, 10, and 20 for each set respectively. The large dataset was already split into train and val folders, therefore training data was all images that were under the train folder. The validation has 20 percent of the images from the val folder and the rest of those images are test data. A validation set was used during the training process to evaluate the models' performance after each epoch. The preprocessed data is saved to perform further model training.

B.2. CNN Model

Regarding the choice of CNN, this project uses ResNet[11], EfficientNet[12], and DenseNet[13] as the pre-defined models. ResNet-18 was chosen for its simplicity and efficiency in training. The architecture employs residual connections, which address the vanishing gradient problem and allow for the training of deeper networks. ResNet-

18, being one of the lighter models in the ResNet family, has only 18 layers, which provides a balance between computational efficiency and performance. Training and validating ResNet-18 are more computationally efficient compared to deeper versions such as ResNet-50 or ResNet-121. For one epoch of training on a medium-sized dataset, ResNet-18 requires significantly fewer floating-point operations per second (FLOPS) and less wall clock time while still achieving high classification accuracy compared to simpler models like VGG. EfficientNet-B0 was selected for its advanced use of compound scaling, which optimizes the model's depth, width, and resolution in a balanced manner. This innovative approach allows EfficientNet to achieve better performance with fewer parameters compared to simple traditional CNNs. EfficientNet-B0, being the baseline model in the EfficientNet family, has an optimal balance between computational cost and accuracy. Despite its simplicity, EfficientNet-B0 outperforms many conventional architectures in terms of accuracy while maintaining a low number of FLOPS and reduced training time per epoch. DenseNet-121 is another model selected for its ability to strengthen feature propagation and encourage feature reuse through densely connected layers. Each layer in DenseNet is directly connected to every other layer, which mitigates the problem of redundant feature learning and promotes efficient parameter usage. While it is slightly more computationally intensive than some of the CNNs, its superior feature reuse makes it an excellent choice for tasks requiring high accuracy. The computational complexity of DenseNet-121 in terms of FLOPS and wall clock time is higher than ResNet-18 but lower than many larger models in its class, making it a suitable choice for achieving a balance between accuracy and computational efficiency. Comparing these models to other available architectures, the selected models have better efficiency and suitability for various tasks. For instance, models like VGG and AlexNet are simpler but also require more computational resources due to a lack of advanced optimizations. With ResNet-18, EfficientNet-B0, and DenseNet-121, this project ensures a balance between computational complexity and accuracy, making these models ideal for the malware image classification task.

B.3. Training Process

Our training process begins with loading the target pre-processed dataset. The input images are stacked into three channels to match the models' expected input format. All target datasets are already divided into training, validation, and testing subsets as mentioned above and PyTorch DataLoader objects are used to efficiently handle data batching and shuffling. After that, the program will load the target pre-defined model. All final fully connected layers in each pre-defined model are adjusted to match the number of classes in the dataset, allowing the model to be fine-tuned

for the specific dataset. If the model is performing transfer learning, the program will also load weights from a .pth file during this load model step. One thing worth mentioning is that we are performing transfer learning on large by load weights that we obtained in the medium dataset and the class numbers in the large dataset and the medium dataset do not match, we explicitly exclude the fully connected layer in the target .pth file. After loading the target model, the program will start to train the model with the provided optimizer, loss function, and hyper-parameter. The training process uses the Adam optimizer, which is a popular choice for deep learning tasks due to its adaptive learning rate capabilities. We chose Adam optimizer because it combines the benefits of momentum-based updates and adaptive learning rates, which can make the training process more efficient. For loss function, this project used cross-entropy loss function, as it effectively measures the difference between predicted probability distributions and the true class labels and it is available in almost all CNN models. Regarding the hyper-parameters, the default value for learning rate is set to $1 * 10^{-4}$, the default value for epochs is 10, and the default value for batch is 32. All hyperparameters are explicitly listed in our code, which allows us to fine-tune our model training to achieve the best results.

After training the model, our program will evaluate the model with several metrics that allow us to assess the effectiveness of our malware detection. Accuracy was used to measure the overall performance of the model. Precision was calculated to evaluate the proportion of correctly identified malware samples (true positives) out of all samples predicted as malware, ensuring the model minimizes false positives. Recall was utilized to measure the model's ability to identify actual malware samples, focusing on minimizing false negatives. To balance these two metrics, we calculated the F1-score, which provides a harmonic mean of precision and recall, offering a comprehensive measure of the model's performance in detecting malware. Additionally, we used the confusion matrix to gain detailed insights into the model's classification performance across all classes, including true positives, true negatives, false positives, and false negatives. To assess the model's ability to discriminate between malware, we evaluated the AUC-ROC. This helped us understand the trade-off between the TPR and the FPR. We also monitored the false positive rate and false negative rate separately to ensure the system minimizes misclassifications, especially in detecting malware where false negatives are critical. These metrics collectively guided us in refining and optimizing our malware detection system to achieve both high accuracy and robust detection capabilities. Additionally to those metrics, we also measured and recorded all training loss, validation loss, training accuracy, and validation accuracy. These information are monitored across epochs, providing insights into the model's learn-

ing process, and allowing us to detect overfitting issues and measure the efficiency of transfer learning. Lastly, our program also plots Grad-CAM images for each class, which helps us slightly understand what the model is "looking at" during decision-making. With these metrics and plots, we perform hyper-parameter tuning using the manual search method. Grid search is more time-consuming and computationally intensive, and random search is unguided which ignores all information provided by metrics. Therefore, even though the manual search method is prone to human error, it is still the best option for this project.

C. Results

C.1. Experiment Setup

Our experiments were designed to first evaluate the performance of three different neural network architectures on one of our target datasets with default hyper-parameters and then continuously manually search for the best set of hyper-parameters based on metrics and plots. Datasets were divided into training, validation, and test sets, and the models were trained and validated using key performance metrics, including precision, recall, confusion matrix, and F1-score, to ensure a comprehensive evaluation of their effectiveness. All hyper-parameters were then carefully tuned based on evaluation. Learning rates were adjusted within a range suitable for convergence without instability. The default learning rate is $1 * 10^{-4}$ and the range of it is between $1 * 10^{-2}$ and $1 * 10^{-6}$. Batch sizes were chosen to balance computational efficiency and gradient stability. The default batch size was 32 and ranged between 16 and 64. The number of epochs was manually adjusted based on observation of the training and validation losses to optimize the training process without overfitting. The default number of epochs was 10 and ranged between 10 and 100. To maintain consistency during the experiment, all models were trained on the same machine with the same hardware and the same processed dataset files.

C.2. Main Results

In this project, we trained, optimized, and evaluated three CNN architectures mixed and matched with three different datasets. In addition, we also performed two transfer learnings that used learned weight from the medium dataset to boost performance, which makes 11 models in total. This section will go through all the main results with figures and make a comparison with other available results. We first trained all selected CNN architectures on a small dataset, as the small dataset only contains two classes, Benign and Malicious. The Benign class contains only Portable Executable (PE) files with no malware in byteplot image format. The Malicious class contains only Portable Executable (PE) files with malware in byteplot im-

age format. As a result, DenseNet, EfficientNet, and ResNet reached 94.58%, 94.88%, and 93.18% training accuracy respectively. Based on training loss/accuracy and validation loss/accuracy plots in Figures 1, 2, and 3, we can see that these three architectures' training accuracy increases consistently across epochs, approaching near-perfect accuracy, and training loss decreases steadily. On the other hand, all three architectures' validation accuracy fluctuates, especially for the ResNet. However, they all generally follow an upward trend, peaking just below their training accuracy. To confirm models do not rely on random or irrelevant features, we also plotted a few Grad-CAM images. Figures 4 and 5 are two examples from ResNet. In the heatmap, regions with warmer colors (yellow/red) indicate higher attention from the model, while cooler colors (blue/green) show less relevance. For Figure 4, the model focuses heavily on the central region of the byteplot, suggesting this area contains distinctive patterns or features critical for Benign. For Figure 5, the model focuses heavily on the lower region of the byteplot, suggesting this region contains distinctive patterns for the model to classify the byteplot image as Malicious. For additional images regarding Grad-CAM or performance information, please visit our GitHub repository[14].

For the moderate dataset, hyper-parameters start to vary for each individual model. DenseNet, EfficientNet, and ResNet reached 96.17%, 97.65%, and 99.80% training accuracy respectively with 17, 20, and 15 epochs. The results obtained from training DenseNet, EfficientNet, and ResNet on the moderate dataset highlight significant differences in their performance metrics. DenseNet121 achieved the highest overall test accuracy of 90.28%, with strong precision, recall, and F1-scores across all classes, demonstrating its robustness in malware classification tasks. EfficientNetB0 followed with an accuracy of 89.16%, showing competitive performance but slightly lower metrics compared to DenseNet121, particularly in the precision and recall values for certain malware classes like "Danabot." ResNet18, on the other hand, delivered a test accuracy of 88.84%, marking the lowest among the three models. Despite this, it maintained reasonable precision and recall values, indicating its capability to classify the malware classes adequately, though with a narrower performance margin. The training time also varied across the models, with DenseNet requiring longer epochs due to its complexity and deeper architecture, as reflected by the wall clock time per epoch. EfficientNet and ResNet were faster in training, with ResNet exhibiting the shortest epoch time, which aligns with its relatively lighter architecture. Our results collectively indicate that while DenseNet provides superior accuracy and robustness, the trade-off comes in the form of increased computational cost. Similar to the small dataset, we also generated Grad-CAM visualizations for the moderate dataset. Figures 6 and 7 are examples from the DenseNet model. These vi-

sualizations demonstrate that our models are effectively extracting features and identifying distinctive patterns, even when working with augmented data. However, due to the complexity of byteplot images, it remains challenging for humans to interpret the specific features and patterns influencing the classification process.

For the large dataset, we retrieved even better results than the moderate dataset. DenseNet, EfficientNet, and ResNet reached 97.89%, 97.86%, and 97.04% training accuracy respectively with 12 epochs. Both ResNet and EfficientNet achieved the highest overall test accuracy of 93.45% with strong precision, recall, and F1-scores across all classes, demonstrating its robustness in malware classification tasks. DenseNet followed with a 92.97% test accuracy, which shows slightly lower accuracy but higher precision and recall in a few specific classes including "Al-laple", "Autorun", "Expiro", and "Injector". Even though ResNet and EfficientNet reached exact same test accuracy, their classification report shows they are good at recognizing different classes. One thing worth noticing is the class "Nashta". It has the lowest precision, recall, and F1-score in all three models across all classes in this dataset. Based on the confusion matrix, we see it got misclassified as "Androm" 14 times. This might caused by feature similarity between "Androm" and "Nashta", insufficient representation of the target class, or overlapping decision boundaries. In general, EfficientNet and ResNet were still faster in training in the large dataset and all three models used the same epoch to reach similar test accuracy. Similar to the moderate dataset, we also generated Grad-CAM visualization for the large dataset but it did not show much information due to the complexity of byteplot images, it challenging for humans to understand the features and patterns used by models. Regarding transfer learning, we performed two transfer learning with EfficientNet and Resnet to classify the large dataset. Loaded weights were learned from the moderate dataset by the same architecture. As a result, EfficientNet achieved 98.31% training accuracy and 95.49% validation accuracy with 10 epochs. ResNet achieved 98.95% training accuracy and 97.22% validation accuracy with the same 10 epochs. The final test accuracy for EfficientNet and ResNet are 95.31% and 96.01%. Based on those outputs, we can see using transfer learning allows models to start training with better accuracy, converge earlier, and use smaller epoch values to get similar performance.

C.3. Ablative Study

The dataset includes 3 different versions, representing large as 20 classes, medium 8 classes, and small datasets as 2 classes. This study examines the impact of both pre-trained and from-scratch model training approaches, as well as optimizer choices, on classification performance. The images were preprocessed into 224x224 grayscale format

and augmented to ensure class balance. The dataset was split into training, validation, and test subsets. It was made sure to have less than 50,000 images per dataset as required by the project description, thus we had a cap of 40,000 images per dataset. The experiments were conducted on a system equipped with an NVIDIA RTX 4060 GPU (16GB RAM, 5.3GHz CPU). This configuration allowed for efficient model training and inference with batch sizes of 32 usually for ResNet-18, DenseNet, and EfficientNet. Using a batch size of 16 stood out to be too slow for such a fast system and size 64 was utilized rarely as the machine allowed to reach that at max. In our ablative experiments, we compared the performance of models with pre-trained weights versus training from scratch. With pre-trained weights, ResNet-18 achieved 92% test accuracy on the large dataset, while EfficientNet reached 90%, and DenseNet performed best at 94%. Training from scratch required more effort: ResNet-18's accuracy peaked at 85% with substantial hyperparameter tuning, EfficientNet reached 83%, and DenseNet struggled, peaking at 88% after 20 epochs. Notably, pre-trained weights enabled faster convergence and higher accuracy, while training from scratch required careful tuning, especially for ResNet on moderate datasets. To be noted that these percentages shared are average approximations of multiple runs and not the best-performing model. For optimizer comparisons, ResNet-18 initially used Adam but was later trained with SGD optimizer at a learning rate of 0.001, momentum of 0.6, and a weight decay of $1e-4$ for smoother convergence, achieving 88% accuracy on the moderate dataset which was previously giving 40 to 70% on the same. DenseNet and EfficientNet were tested with both Adam ($lr=0.0001$) and SGD; Adam provided faster convergence but was more sensitive to learning rate changes, reaching 90% accuracy on the moderate dataset around 15 epochs.

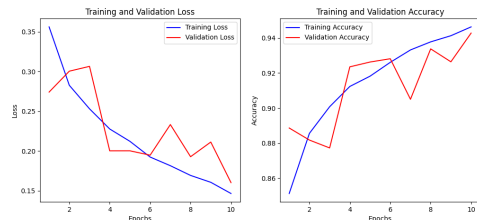


Figure 1. Accuracy and loss plot from the DenseNet121 model.

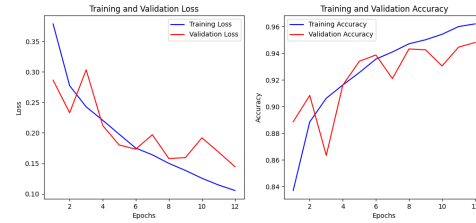


Figure 2. Accuracy and loss plot from the EfficientNetB0 model.

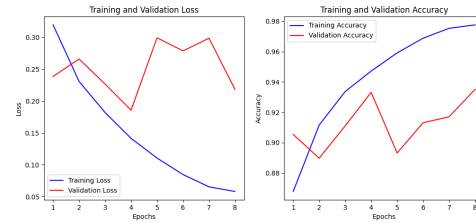


Figure 3. Accuracy and loss plot from the ResNet18 model.

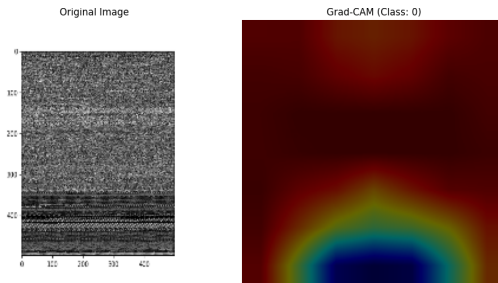


Figure 4. Grad-CAM example for Benign file.

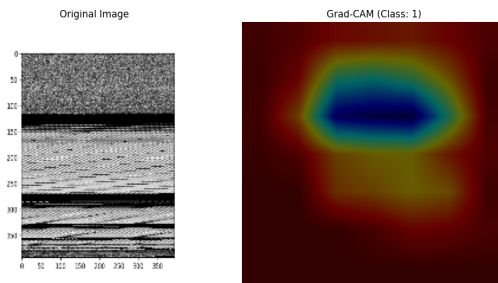


Figure 5. Grad-CAM example for Malicious file.

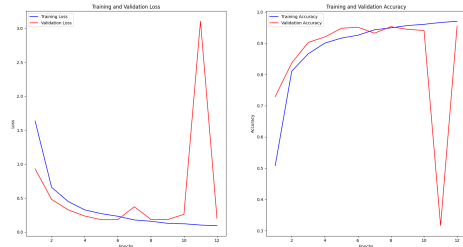


Figure 6. Accuracy and loss plot from the DenseNet121 model.

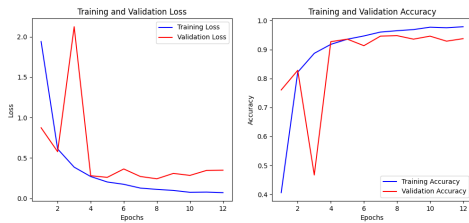


Figure 7. Accuracy and loss plot from the EfficientNetB0 model.



Figure 8. Accuracy and loss plot from the ResNet18 model.

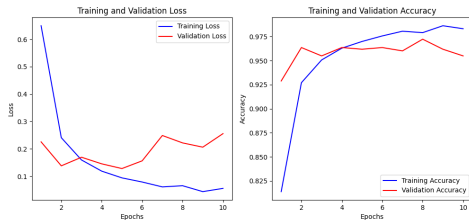


Figure 9. Accuracy and loss plot from the EfficientNetB0 model with transfer learning.

References

- [1] Djenna A, Bouridane A, Rubab S, Marou IM. *Artificial Intelligence-Based Malware Detection, Analysis, and Mitigation*. Symmetry. 2023; 15(3):677. <https://doi.org/10.3390/sym15030677>
- [2] Molinari S, Packer J. *MALWARE SCIENCE a Comprehensive Guide to Detection, Analysis, and Compliance*. Packt Publishing Ltd.; 2023. <https://www.packtpub.com/product/malware-science-a-comprehensive-guide-to-detection-analysis-and-compliance/9781804618646>

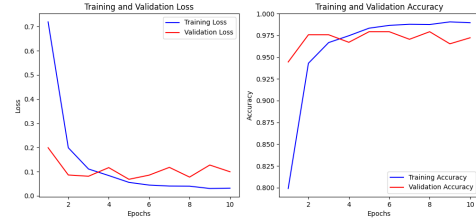


Figure 10. Accuracy and loss plot from the ResNet18 model with transfer learning.

- [//www.oreilly.com/library/view/-/9781804618646/](https://www.oreilly.com/library/view/-/9781804618646/)
- [3] Wolsey, A. (2022). *The State-of-the-Art in AI-Based Malware Detection Techniques: A Review*. arXiv preprint arXiv:2210.11239. <https://arxiv.org/abs/2210.11239>
 - [4] Dixon Natasha. (2023). *The Role of AI in Malware Detection and Prevention* MalwareBrains. Available online: <https://malwarebrains.com/ai-in-malware-detection/>
 - [5] A. Almuqren, M. Frikha and A. Albuali. (2023). *Automated Malware Detection Based on a Machine Learning Algorithm*. 2023 IEEE Tenth International Conference on Communications and Networking (ComNet), Hammamet, Tunisia, 2023, pp. 1-12, doi: 10.1109/ComNet60156.2023.10366550. <https://ieeexplore.ieee.org/document/10366550>
 - [6] Gaurav Pendharkar. (2022). *Blended Malware Image Dataset*. Kaggle. Available online: <https://www.kaggle.com/datasets/gauravpendharkar/blended-malware-image-dataset>
 - [7] Walt. (2022). *Malware Images*. Kaggle. Available online: <https://www.kaggle.com/datasets/walt30/malware-images/>
 - [8] Ajit Varma. (2024). *Malware-Byteplot Images*. Kaggle. Available online: <https://www.kaggle.com/datasets/ajit22/malware-byteplot-images>
 - [9] Ghasemi A, Soleymani M, Asgarian F. (2022). *A novel deep learning-based approach for image classification using a hybrid feature extraction method*. Mathematics. 2022;14(11):2304. doi:10.3390/math14112304. Available from: <https://www.mdpi.com/2073-8994/14/11/2304>

- [10] Poulson JM, Wu YC, Nadiga BT. (2022). *Normalization effects on deep neural networks*. arXiv; 2022. Available from: <https://ar5iv.labs.arxiv.org/html/2209.01018>
- [11] He, K., Zhang, X., Ren, S., & Sun, J. (2016). *Deep residual learning for image recognition*. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 770-778).
- [12] Tan, M., & Le, Q. (2019). *EfficientNet: Rethinking model scaling for convolutional neural networks*. In International Conference on Machine Learning (pp. 6105-6114). PMLR.
- [13] Huang, G., Liu, Z., Van Der Maaten, L., & Weinberger, K. Q. (2017). *Densely connected convolutional networks*. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 4700-4708).
- [14] habeebdashti. (2021). *COMP6721-GroupA*. Github. 2024. Available from: <https://github.com/habeebdashti/COMP6721-GroupA>

702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755